

PRÁCTICA 3

Mario VILAR
20392724

David Díez
20392643

1 de noviembre de 2023



UNIVERSITAT DE
BARCELONA

ÍNDICE

1	Introducción	2	<i>Ejercicio 9</i>	3
2	Preguntas	2	<i>Ejercicio 10</i>	3
	<i>Ejercicio 1</i>	2	<i>Ejercicio 11</i>	4
	<i>Ejercicio 2</i>	2	<i>Ejercicio 12</i>	4
	<i>Ejercicio 3</i>	2	<i>Figura 2.1</i>	4
	<i>Ejercicio 4</i>	2	<i>Ejercicio 13</i>	4
	<i>Ejercicio 5</i>	3	<i>Ejercicio 14</i>	5
	<i>Ejercicio 6</i>	3	<i>Ejercicio 15</i>	5
	<i>Ejercicio 7</i>	3	3 Conclusiones	5
	<i>Ejercicio 8</i>	3	Referencias	5

INTRODUCCIÓN

En esta práctica, exploraremos detalladamente varios comandos y herramientas de Linux que permiten interactuar con los sistemas de archivos. Profundizaremos en cómo estos comandos facilitan la administración y comprensión del funcionamiento interno de los sistemas de archivos, habilidades esenciales para cualquier profesional de la informática que busque sobresalir en el campo tecnológico.

PREGUNTAS

Ejercicio 1. ¿Qué comando has utilizado? Describe cada una de las columnas de la salida de este comando.

Demostración. `df` se utiliza para conocer el espacio libre en un `filesystem` concreto.

1. Filesystem (Sistema de archivos): nombre del sistema de archivos montado.
2. Size: tamaño total del sistema en un formato legible por humanos, generalmente en gigabytes (GB).
3. Used: cantidad de espacio del sistema de archivos *que está siendo utilizada*, también en un formato legible por humanos.
4. Avail: espacio que aún no ha sido utilizado.
5. Use %: porcentaje de espacio en el sistema de archivos que está siendo utilizado. .
6. Mounted on: ubicación en el sistema de archivos donde el dispositivo se encuentra montado. ✓

Con el comando `df -Th --total` obtenemos una fila con totales. Fijémonos que el tamaño de la carpeta compartida con el host nos devuelve el tamaño del disco de nuestro ordenador (que no MV).

Ejercicio 2. ¿Cuál es el tamaño total del disco en bytes a partir del número de sectores y el tamaño de cada sector?

Demostración. Con el ejercicio siguiente, 3, podemos saber el tamaño total del disco en bytes a partir del número de sectores y el tamaño de cada sector. En efecto, la suma total de sectores por particiones es $2048 + 4194304 + 58714112 = 62910464$. El tamaño de cada sector es de $512B$. Por tanto:

$$\text{sectores} \cdot \text{tamaño} = 62910464 \cdot 512 = 32210157568 \text{ Bytes} = 29,998 \text{ GiB.} \quad (2.1)$$

En efecto, (2.1) nos devuelve el tamaño de disco asignado a nuestra máquina virtual. ✓

Ejercicio 3. ¿Cuántas particiones tiene el disco y cuáles son sus números de inicio y fin de los sectores en cada partición?

Demostración. A partir de la tabla de particiones (`sudo fdisk /dev/sda` desde el usuario `oslabadmin` con `su`) vemos que las particiones del disco son `sda1`, `sda2`, `sda3`. Su respectiva tabla es:

Device	Start	End	Sectors	Size
/dev/sda1	2048	4095	2048	1M
/dev/sda2	4096	4198399	4194304	2G
/dev/sda3	4198400	62912511	58714112	28G

Cuadro 1: Tabla del ejercicio 3.

Y con esto ya hemos acabado. ✓

Ejercicio 4. ¿Qué tipos de configuraciones de particiones y sistemas de archivos existen en la actualidad? Describe dos estrategias.

Demostración. Podemos ver los diferentes tipos de configuraciones para sistemas de archivos que hay a través de comandos ya vistos en esta práctica (bajo la columna `type` del comando `df -hT`). Se usan, por ejemplo, los sistemas `tmpfs` y `ext4`. Los vamos a explicar brevemente:

1. **ext4**: sistema de archivos usado principalmente en Linux. Destaca su alto rendimiento, sobre todo respecto sus versiones anteriores.
2. **tmpfs**: sistema de archivos temporal, limitado a la memoria RAM de un sistema Linux o Unix. Extremadamente rápido, pero volátil.

A su vez, con el ejercicio 5 vemos que hay un gran número de tipo de particiones, y no nos entretendremos a exponerlos aquí. Comentar, por ejemplo, que hay uno para EFI (usada para el arranque del sistema), otro **swap** y el más recurrente en nuestra MV: **Linux Filesystem**. ✓

Ejercicio 5. Utiliza el comando **cfdisk** para crear una nueva partición pequeña en el disco **/dev/sda** y verifica que se ha creado correctamente con el comando **fdisk**.

Demostración. Hecho el cambio a **oslabadmin** con el comando correspondiente, no hay ningún problema para crear la partición deseada.

1. Primero liberamos espacio de **sda3**, la partición con más espacio de la MV.
2. Después, utilizamos ese espacio para crear una nueva partición, **sda4**, con **cfdisk**. Escribimos los cambios en la tabla de particiones, muy importante.
3. Usando los recursos del ejercicio 3 vemos que la partición ha sido creada, con la medida especificada en el paso 1. ✓

Ejercicio 6. Continuando con las líneas de comandos, ¿cómo podrías listar/contar los ficheros de tamaño mayor de 100Mb dentro de ese directorio?

Demostración. No existe ninguna manera de hacerlo con los comandos provistos hasta ahora en esta práctica. Nos hemos inclinado por hacerlo con un comando que ya vimos en SOI. La línea en cuestión ha sido ejecutada desde el directorio **usr**:
find . -type f -size +100M. Si los quisiéramos contar, podríamos usar una tubería con el comando **wc -l**. ✓

Ejercicio 7. ¿Qué representa un dispositivo **loop** en la salida de **lsblk** y para qué se utiliza comúnmente?

Demostración. La información proporcionada por **man lsblk** es insuficiente, después de una búsqueda vemos que **loop** sirve para montar archivos como si fueran sistemas de archivos completos, y puede servir para montar ciertos archivos como imágenes de disco. Por cierto, **lsblk** es otra manera de comprobar que la partición creada en el ejercicio 5 ha sido satisfactoria. ✓

Ejercicio 8. ¿Cómo identificarías un disco duro (HDD) y una unidad de estado sólido (SSD) utilizando **lsblk**?

Demostración. Para identificar el HDD o el SSD utilizamos el comando **lsblk -o NAME, ROTA**. Con esto obtenemos una tabla con el nombre de los bloques del sistema y un **r** para indicar si es HDD o **o** en caso de SSD. En nuestra máquina virtual hemos obtenido todo **r**, esto quiere decir que se utiliza un disco duro HDD. ✓

Ejercicio 9. ¿Cuál es la capacidad máxima teórica de un volumen en un sistema de archivos **ext4**, y cuál es el tamaño máximo teórico de un archivo individual cuando se utiliza un tamaño de bloque estándar de 4 KB?

Demostración. El tamaño máximo teórico del volumen con el sistema de archivos **ext4** es de 1 exabyte (1 EB), según [Wik23]. A su vez, cuando se utiliza un tamaño de bloque estándar de 4 KB (4096 bytes), el tamaño máximo de un archivo individual es de aproximadamente 16 terabytes (16 TB). En efecto, el tamaño máximo teórico de un archivo individual en un sistema de archivos **ext4** está relacionado con el tamaño del bloque utilizado. ✓

Ejercicio 10. ¿En qué directorio de nivel superior encontrarías estos archivos y por qué es el lugar adecuado para hacer una copia de seguridad? Explica brevemente.

Demostración. El directorio de nivel superior más adecuado para hacer una copia de seguridad es **/etc**. Dicho directorio almacena los archivos de configuración del sistema, y está pensado para facilitar la identificación y copia de archivos de configuración. Además, este directorio está protegido contra accesos no autorizados lo que da un nivel de seguridad extra a la copia de seguridad. ✓

Ejercicio 11. Indica que *i-node* tiene el archivo `test.txt` y como lo has averiguado. Muestra el nombre del archivo, el tamaño en kilobytes (KB) y la fecha de última modificación del archivo creado anteriormente (`test.txt`). Personaliza la salida para que el tamaño se muestre en KB y la fecha esté en formato año-mes-día hora:minuto:segundo.

Demostración. Hemos creado un archivo con `touch` y hemos añadido un `lorem ipsum` de 4 párrafos. Acto seguido, sin cambiar de directorio, utilizamos el comando `stat test.txt` y obtenemos los datos del fichero. Entre ellos tenemos el nombre `test.txt`, el tamaño en bytes del archivo, 1475. También obtenemos el número de bloques y, efectivamente, también el *i-node*, que en este caso es el 656342. Para mostrar el tamaño en kilobytes (que no kibibytes, diferencia importante) y la última fecha de modificación en el formato deseado utilizamos el siguiente script (puesto de forma compacta para ocupar menos):

```
1 #!/bin/bash
2 nombre=$(stat -c %n "$1"); size=$(stat -c %s "$1"); kb=$((size/1000))
3 fecha=$(stat -c %y "$1" | awk '{print $1, $2}') # no nos interesa tercera columna, que es la zona horaria
4 echo "$nombre"; echo "Tamaño: $kb KB"; echo "Fecha última modificación: $fecha"
```

y obtenemos

```
hello.txt
Tamaño: 3 KB
Fecha última modificación: 2023-10-29 14:29:46.202269660
```

que es lo que deseamos. Notar que Bash no trabaja con decimales y trunca los KB al entero inferior. ✓

Ejercicio 12. ¿Cuál es el número de *i-nodes* del sistema de archivo y cuantos quedan libres sin usar? Elimina el archivo `test.txt` y mira de nuevo el número de *i-nodes* libres. ¿Ha cambiado?

Demostración. Para ver la cantidad de *i-nodes* del sistema del archivo tenemos que ejecutar el comando `df -i`. El resultado que obtenemos es una tabla que contiene el nombre del sistema y tres columnas: una para la cantidad total de *i-nodes*, otra para la cantidad que están en uso y otra para los que hay libres. Abreviaremos `/dev/mapper/ubuntu--vg-ubuntu--lv por mapper`. En particular, los valores que obtenemos son:

Filesystem	Inodes	IUsed	IFree
tmpfs	376804	1062	375742
mapper	917504	203506	713998
tmpfs	376804	1	376803
tmpfs	376804	5	376799
/dev/sda2	131072	322	130750
shared-folder	1000	-999000	1000000
tmpfs	75360	142	75218

Cuadro 2: *i-nodes* antes de eliminar el archivo

Filesystem	Inodes	IUsed	IFree
tmpfs	376804	1061	375743
mapper	917504	231792	685712
tmpfs	376804	1	376803
tmpfs	376804	5	376799
/dev/sda2	131072	323	130749
shared-folder	1000	-999000	1000000
tmpfs	75360	141	75219

Cuadro 3: *i-nodes* después de eliminar el archivo

Figura 2.1: Izquierda: Comando `df -i`. Derecha: Después de eliminar el archivo utilizando `rm test.txt` y al volver a ejecutar `df -i`.

En efecto, se han reducido los *i-nodes* usados: de la primera y última `tmpfs`, el `mapper` cambia radicalmente. En el `sda2` se produce el paso contrario: aumentan los *i-nodes*. ✓

Ejercicio 13. Describe el proceso que se detalla a continuación

Demostración. La partición ya la hemos creado en el ejercicio 5 y por lo tanto no hace falta crearla de nuevo. Para todo el resto, el código que hemos utilizando ha sido

```

sudo mkfs -t ext4 /dev/sda4      # Crear un sistema de archivos ext4
sudo mkdir /mnt/prueba          # Crear un directorio de punto de montaje
sudo mount /dev/sda4 /mnt/prueba # y montar la partición
sudo umount /mnt/pruebas        # Desmontar la partición
sudo fsck -n /dev/sda4          # Verificación simulada del sistema de archivos
sudo mount /dev/sda4 /mnt/prueba # Remontar el sistema de archivos

```

En particular, durante la ejecución hemos obtenido el mensaje de

```

fsck from util-linux 2.37.2
e2fsck 1.46.5 (30-Dec-2021)
/dev/sda4: clean, 11/65408 files, 8851/261627 blocks

```

al ejecutar el comando `sudo fsck -n /dev/sda4`. En caso de tener un error, por ejemplo, intentar ejecutar el comando sobre una partición en uso obtendríamos un **Warning!** de que `/dev/sda4 is in use`. ✓

Ejercicio 14. ¿Cómo cambiarías el nombre del volumen de un sistema de archivos (por ejemplo `/dev/sda4`) por otro (por ejemplo `/dev/S02`)?

Demostración. El código que hemos utilizado para cambiar el nombre del volumen es

```

sudo umount /dev/sda4      # Desmontamos el nombre
sudo tune2fs -L S02 /dev/sda4 # Cambiamos el nombre
sudo mount /dev/sda4 /mnt/pruebas # Volvemos a montar

```

Una vez ejecutado el comando `sudo tune2fs -L S02 /dev/sda4` obtenemos el mensaje `tune2fs 1.46.5 (30-Dec-2021)` y para comprobar ejecutamos el comando `sudo e2label /dev/sda4`. Efectivamente obtendremos el mensaje `S02`. ✓

Ejercicio 15. ¿Qué ha pasado? Describe el comportamiento, así como la respuesta del `e4defrag`.

Demostración. Primero de todo, el código necesario para realizar este proceso es

```

dd if=/dev/zero of=large_file.txt bs=1M count=1000
filefrag large_file.txt

```

Hasta aquí la salida para nuestro archivo generado de 1GB es *24 extents found*¹, parámetro que mide la fragmentación. Tras la creación del archivo es probable que esté fragmentado, lo que significa que está disperso en partes por el disco, lo que puede ralentizar el acceso a los datos. A continuación (ejecución de `filefrag`), se espera que las partes del archivo estén más contiguas en el disco, lo que mejoraría el acceso y reduciría la fragmentación.

```

sudo e4defrag large_file.txt
filefrag large_file.txt

```

Después de desfragmentar el archivo con `e4defrag` vemos que el nombre de *extents* se ha reducido en una unidad. En efecto, el comando `filefrag` mostrará una situación con menos fragmentación. ✓

3

CONCLUSIONES

En esta práctica hemos aprendido que los sistemas de archivos son pilares fundamentales en el mundo de la informática, proporcionando una estructura organizativa para almacenar, acceder y gestionar datos en dispositivos de almacenamiento. Mario se ha centrado en los problemas de la primera semana; David, en los de la segunda.

REFERENCIAS

[Wik23] Wikipedia. *Ext4 — Wikipedia, La enciclopedia libre*. [Internet; descargado 2-octubre-2023]. 2023. URL: <https://es.wikipedia.org/w/index.php?title=Ext4&oldid=154241908>.

¹ Un extent es un conjunto de bloques físicos contiguos, mejorando el rendimiento al trabajar con ficheros de gran tamaño y reduciendo la fragmentación.